
CS 201, Spring 2026

Homework Assignment 1

Due: 23:59, March 6, 2026

1 Introduction

Welcome to Bilkent Academy of Wizardry!

In this assignment, you will design and implement a wizard school potion management system called **WizardPotionInventorySystem**. At Bilkent Academy of Wizardry, student wizards prepare for the toughest weeks of the semester by brewing potions. Each **student wizard** has a personal **potion inventory** that stores the potions they currently own. Each potion has a **potion name** and a **strength** value.

In your implementation, you MUST use dynamically allocated arrays for storing:

- **student wizards** in the system, and
- **potions** in each student wizard's potion inventory.

You will implement the following functionalities (details are given in Section 2):

1. Add a student wizard to the academy.
2. Remove a student wizard from the academy.
3. Brew (add) a potion to a student wizard's potion inventory.
4. Discard (remove) a potion from a student wizard's potion inventory.
5. Transfer a potion from one student wizard to another.
6. Show the list of all student wizards.
7. Show detailed information about a student wizard.
8. Show detailed information about a potion.

2 Implementation Details

2.1 Data Model

The system manages the following concepts:

- **Student Wizard:** Identified by a unique **name** (string) in the system, and has a **house** (string). Each student wizard owns a dynamically allocated array representing their potion inventory.
- **Potion:** Each potion has a **potionName** (string) and a **strength** (integer).

Uniqueness Rules:

- Student wizard names are **unique** in the system (case-sensitive).
- Within a given student wizard's potion inventory, **potionName is unique** (case-sensitive). However, different student wizards may possess potions with the same name.

2.2 Functions

The member functions of `WizardPotionInventorySystem` are explained as follows:

- `addStudentWizard(const string name, const string house)`
Adds a new student wizard with the given `name` and `house`. If a student wizard with the same name already exists, the function should display a warning message. Initially, a student wizard has an empty potion inventory.
- `removeStudentWizard(const string name)`
Removes the specified student wizard from the system. If the student wizard does not exist, the function should display a warning message. If the student wizard has one or more potions in their potion inventory, all of those potions are discarded (removed) and the student wizard is removed successfully.
- `brewPotion(const string studentName, const string potionName, const int strength)`
Adds a new potion to the potion inventory of the specified student wizard. If the student wizard does not exist, the operation fails and a warning message is displayed. If a potion with the same `potionName` already exists in that student's potion inventory, the operation fails and a warning message is displayed. Otherwise, the potion is added to the end of the student's potion inventory. Different students can have the same potion with different strengths.
- `discardPotion(const string studentName, const string potionName)`
Removes the potion with `potionName` from the specified student's potion inventory. If the student wizard does not exist, the operation fails and a warning message is displayed. If the potion does not exist in that student's potion inventory, the operation fails and a warning message is displayed.
- `transferPotion(const string potionName, const string fromStudent, const string toStudent)`
Transfers a potion (identified by `potionName`) from one student wizard to another. If at least one of the student wizards does not exist, the operation fails and a warning message is displayed. If the potion does not exist in the `fromStudent`'s potion inventory, the operation fails and a warning message is displayed. If a potion with the same `potionName` already exists in the `toStudent`'s potion inventory, the transfer is not allowed and a warning message is displayed. Otherwise, the potion is removed from `fromStudent`'s inventory and appended to the end of `toStudent`'s inventory.
- `showAllStudentWizards() const`
Displays a list of all student wizards in the system. For each student wizard, print their name, house, the number of potions in their potion inventory, and the total potion strength (sum of strengths). **Order matters:** Wizards must be displayed in alphabetical order according to their names.

- `showStudentWizard(const string name) const`
Displays detailed information about the specified student wizard, including their name, house, number of potions, total potion strength, and the potions in their potion inventory. If the student wizard does not exist, the function should display a warning message. **Order matters:** Potions of the wizard must be displayed in the order they were brewed/added to that student.
- `showPotion(const string potionName) const`
Searches for a potion with name `potionName` across **all** student wizards. If found, displays the potion's name, the name of all student wizards who own it, and the corresponding strengths. If no such potion exists in the system, the function should display a warning message. **Order matters:** Wizards who have the potion must be displayed in alphabetical order.

2.3 Class Implementation

Your solution must be implemented in a class called **WizardPotionInventorySystem**. The interface for the class must be written in a file called **WizardPotionInventorySystem.h** and its implementation must be written in a file called **WizardPotionInventorySystem.cpp**. You can define additional public and private member functions and data members in this class. You can also define additional classes in your solution and implement them in separate files.

```

1 class WizardPotionInventorySystem {
2 public:
3     WizardPotionInventorySystem();
4     ~WizardPotionInventorySystem();
5
6     void addStudentWizard(const string name, const string house);
7     void removeStudentWizard(const string name);
8
9     void brewPotion(const string studentName,
10                    const string potionName,
11                    const int strength);
12
13     void discardPotion(const string studentName,
14                       const string potionName);
15
16     void transferPotion(const string potionName,
17                        const string fromStudent,
18                        const string toStudent);
19
20     void showAllStudentWizards() const;
21     void showStudentWizard(const string name) const;
22     void showPotion(const string potionName) const;
23 };

```

2.4 Example Test Case and Output

Here is an example test program that uses this class and the corresponding output. We will use a similar program to test your solution so make sure that the name of the class is **WizardPotionInventorySystem**, its interface is in the file called **WizardPotionInventorySystem.h**, and the required functions are defined as shown above. Your implementation MUST use

EXACTLY the same format given in the example output to display the messages expected as the result of the defined functions.

Example test code:

```
1
2 #include <iostream>
3 #include "WizardPotionInventorySystem.h"
4 using namespace std;
5
6 int main()
7 {
8     WizardPotionInventorySystem sys;
9
10    sys.showAllStudentWizards();
11    cout << endl;
12
13    sys.showStudentWizard("Ron");
14    sys.showPotion("Polyjuice Potion");
15    cout << endl;
16
17    sys.addStudentWizard("Harry", "Gryffindor");
18    sys.addStudentWizard("Hermione", "Gryffindor");
19    sys.addStudentWizard("Draco", "Slytherin");
20    sys.addStudentWizard("Luna", "Ravenclaw");
21
22    sys.addStudentWizard("Harry", "Slytherin");
23    cout << endl;
24
25    sys.brewPotion("Snape", "Felix Felicis", 10);
26    sys.brewPotion("Harry", "Polyjuice Potion", 6);
27    sys.brewPotion("Harry", "Healing Draught", 4);
28    sys.brewPotion("Draco", "Healing Draught", 5);
29    sys.brewPotion("Luna", "Healing Draught", 10);
30
31    sys.brewPotion("Harry", "Polyjuice Potion", 8);
32    sys.brewPotion("Hermione", "Polyjuice Potion", 9);
33    sys.brewPotion("Draco", "Sleeping Potion", 5);
34    cout << endl;
35
36    sys.showAllStudentWizards();
37    cout << endl;
38
39    sys.showStudentWizard("Harry");
40    cout << endl;
41
42    sys.showPotion("Healing Draught");
43    cout << endl;
44
45    sys.showPotion("Invisibility Potion");
46    cout << endl;
47
48    sys.discardPotion("Luna", "Dream Potion");
49    sys.discardPotion("Ghost", "Anything");
50    sys.discardPotion("Draco", "Sleeping Potion");
51    sys.discardPotion("Draco", "Sleeping Potion");
52    cout << endl;
```

```

53
54     sys.removeStudentWizard("Draco");
55     sys.removeStudentWizard("Draco");
56     sys.removeStudentWizard("Luna");
57     cout << endl;
58
59     sys.addStudentWizard("Neville", "Gryffindor");
60     cout << endl;
61
62     sys.transferPotion("Polyjuice Potion", "Ghost", "Neville");
63     sys.transferPotion("Polyjuice Potion", "Harry", "Ghost");
64     sys.transferPotion("Felix Felicis", "Harry", "Neville");
65     cout << endl;
66
67     sys.brewPotion("Neville", "Polyjuice Potion", 1);
68     sys.transferPotion("Polyjuice Potion", "Harry", "Neville");
69     cout << endl;
70
71     sys.transferPotion("Healing Draught", "Harry", "Neville");
72     sys.transferPotion("Healing Draught", "Harry", "Neville");
73     cout << endl;
74
75     sys.showStudentWizard("Hermione");
76     cout << endl;
77
78     sys.showStudentWizard("Neville");
79     cout << endl;
80
81     sys.showAllStudentWizards();
82     return 0;
83 }

```

Output of the example test code:

```

1 Student wizards in the system:
2 None
3
4 Student wizard Ron does not exist.
5 Potion Polyjuice Potion does not exist.
6
7 Added student wizard Harry.
8 Added student wizard Hermione.
9 Added student wizard Draco.
10 Added student wizard Luna.
11 Cannot add student wizard. Student wizard Harry already exists.
12
13 Cannot brew potion. Student wizard Snape does not exist.
14 Brewed potion Polyjuice Potion for student wizard Harry.
15 Brewed potion Healing Draught for student wizard Harry.
16 Brewed potion Healing Draught for student wizard Draco.
17 Brewed potion Healing Draught for student wizard Luna.
18 Cannot brew potion. Potion already exists in potion inventory of Harry.
19 Brewed potion Polyjuice Potion for student wizard Hermione.
20 Brewed potion Sleeping Potion for student wizard Draco.
21

```

```

22 Student wizards in the system:
23 Draco, House: Slytherin, 2 potion(s), 10 total strength.
24 Harry, House: Gryffindor, 2 potion(s), 10 total strength.
25 Hermione, House: Gryffindor, 1 potion(s), 9 total strength.
26 Luna, House: Ravenclaw, 1 potion(s), 10 total strength.
27
28 Student wizard:
29 Harry, House: Gryffindor, 2 potion(s), 10 total strength.
30 Potions:
31 Polyjuice Potion, strength 6.
32 Healing Draught, strength 4.
33
34 Potion "Healing Draught" found in 3 student wizard(s):
35 1. Draco, strength 5.
36 2. Harry, strength 4.
37 3. Luna, strength 10.
38
39 Potion Invisibility Potion does not exist.
40
41 Cannot discard potion. Potion does not exist in potion inventory of Luna.
42 Cannot discard potion. Student wizard Ghost does not exist.
43 Discarded potion Sleeping Potion from student wizard Draco.
44 Cannot discard potion. Potion does not exist in potion inventory of Draco.
45
46 Removed student wizard Draco.
47 Cannot remove student wizard. Student wizard Draco does not exist.
48 Removed student wizard Luna.
49
50 Added student wizard Neville.
51
52 Cannot transfer potion. One or both student wizards do not exist.
53 Cannot transfer potion. One or both student wizards do not exist.
54 Cannot transfer potion. Potion does not exist in potion inventory of Harry.
55
56 Brewed potion Polyjuice Potion for student wizard Neville.
57 Cannot transfer potion. Potion already exists in potion inventory of Neville.
58
59 Transferred potion Healing Draught from Harry to Neville.
60 Cannot transfer potion. Potion does not exist in potion inventory of Harry.
61
62 Student wizard:
63 Hermione, House: Gryffindor, 1 potion(s), 9 total strength.
64 Potions:
65 Polyjuice Potion, strength 9.
66
67 Student wizard:
68 Neville, House: Gryffindor, 2 potion(s), 5 total strength.
69 Potions:
70 Polyjuice Potion, strength 1.
71 Healing Draught, strength 4.
72
73 Student wizards in the system:
74 Harry, House: Gryffindor, 1 potion(s), 6 total strength.
75 Hermione, House: Gryffindor, 1 potion(s), 9 total strength.
76 Neville, House: Gryffindor, 2 potion(s), 5 total strength.

```

3 Specifications

1. You ARE NOT ALLOWED to modify the given parts of the header file. **You MUST use dynamically allocated arrays with only the necessary amount of memory in your implementation.** That is, if there are N student wizards in the system, you should allocate memory only for those N student wizards. Similarly, each student wizard's potion inventory must use memory only for the potions they currently own. **You will get no points if you use fixed-sized arrays, linked lists, or any other data structures such as vectors/arrays/etc. from the standard library.** However, if necessary, you may define additional data members and member functions.
2. Moreover, you ARE NOT ALLOWED to use any global variables or any global functions.
3. Output message for each operation MUST match the format shown in the output of the example code. Your output will be compared verbatim with the expected output during evaluation.
4. Your code MUST NOT have any memory leaks. You will lose points if you have memory leaks in your program even though the outputs of the operations are correct. To detect memory leaks, you may want to use Valgrind which is available at <http://valgrind.org>.
5. You may assume that the inputs for the functions are in correct format so that you do not need to make any input checks (e.g., strength values will be valid integers).
6. Student wizard names and potion names are **unique and case-sensitive** according to the rules described above. However, different student wizards may possess potions with the same name.

4 Submission

1. In this assignment, you must have separate interface and implementation files (i.e., separate `.h` and `.cpp` files) for your class. Your class name MUST BE `WizardPotionInventorySystem` and your file names MUST BE `WizardPotionInventorySystem.h` and `WizardPotionInventorySystem.cpp`. Note that you may write additional class(es) in your solution.
2. The code (`main` function) given above is just an example. We will test your implementation using different scenarios, which will contain different function calls. Thus, do not test your implementation only by using this example code. We recommend you to write your own driver files to make extra tests. However, you MUST NOT submit these test codes (we will use our own test code). In other words, do not submit a file that contains a function called `main`.
3. You should put all of your `.h` and `.cpp` files into a folder and zip the folder (in this zip file, there should not be any file containing a `main` function). The name of this zip file should conform to the following name convention: `secX-Firstname-Lastname-StudentID.zip` where `X` is your section number. The submissions that do not obey these rules will not be graded. Please do not use Turkish letters in your file and folder names.
4. Make sure that each file that you submit (each and every file in the archive) contains your name, section, and student number at the top as comments.

5. You are free to write your programs in any environment (you may use Linux, Windows, MacOS, etc.). On the other hand, we will test your programs on “dijkstra.ug.bcc.bilkent.edu.tr” and we will expect your programs to compile and run on the dijkstra machine. Your code will be tested by using an automated test suite that includes multiple test cases where each case corresponds to a specific number of points in the overall grade. We will provide you with example test cases by email. Thus, we strongly recommend you to make sure that your program successfully compiles and correctly works on dijkstra.ug.bcc.bilkent.edu.tr before submitting your assignment. If your current code does not fully compile on dijkstra before submission, you can try to comment out the faulty parts so that the remaining code can be compiled and tested during evaluation.
6. This assignment is due by 23:59 on March 6, 2026. You should upload your work to Moodle before the deadline. No hardcopy submission is needed. Late submissions will not be accepted (if you can upload to Moodle, then you are fine). There will be no extension to this deadline.
7. We use an automated tool as well as manual inspection to check your submissions against plagiarism. For questions regarding academic integrity and use of external tools (including generative AI tools), please refer to the course home page and the Honor Code for Introductory Programming Courses (CS 101/102/201/202) at https://docs.google.com/document/d/1v_3ltpV_1C1LsROXrMbojyuv4KrFQAm1uoz3SdC-7es/edit?usp=sharing.
8. This homework will be graded by your TAs **Sude Önder** (sude.onder@bilkent.edu.tr) and **Pınar Gül** (pinar.gul@bilkent.edu.tr). Thus, you may ask your homework related questions directly to them. There will also be a forum on Moodle for questions.